PYTHON ACTIVITY 1

CASE STUDIES 21–30

•Problem Statement
• Objectives
• Assumptions
• Algorithm
• Pseudocode
•Python Source Code
• Test Cases
• Output Screenshots
• Conclusion

QUESTION 21: Traffic Violation Penalty Calculator

1. PROBLEM STATEMENT

Process multiple offences, calculate their total fines and check a sample disqualification condition.

2. OBJECTIVES

Read vehicle and offence count • Find each base fine • Add repeat-offence surcharge • Add fines and check disqualification • Display total penalty and status

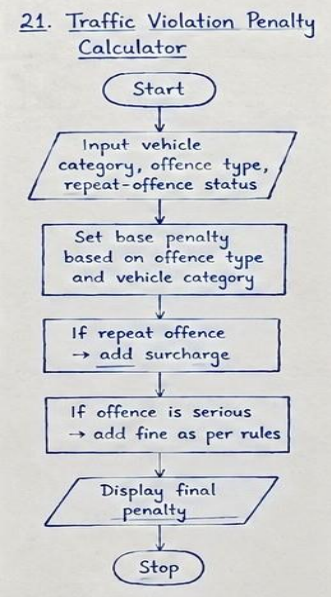
3. ASSUMPTIONS

speeding=Rs.1000; signal=Rs.1500; no_helmet=Rs.500. Repeat adds Rs.500. Signal offence or total >= Rs.3000 flags disqualification. Sample rules only.

4. ALGORITHM

1. Start → 2. Input vehicle and offence count → 3. Read offence and repeat status → 4. Find fine and add surcharge → 5. Add fine to total → 6. Signal offence or total >= 3000? → 7. Display penalty and status → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input vehicle and offence count
3. Read offence and repeat status
4. Find fine and add surcharge
5. Add fine to total
6. Signal offence or total >= 3000?
```

```
7. Display penalty and status
8. Stop
END
```

7. PYTHON SOURCE CODE

```
rules={"speeding":1000,"signal":1500,"no_helmet":500}
total=0 disqualified=False vehicle=input("Vehicle
(car/bike): ").lower() n=int(input("Number of
offences: ")) for i in range(n):
    offence=input("Offence: ").lower()
repeat=input("Repeat? (yes/no): ").lower()
if offence not in rules:
        print("Invalid offence; skipped")
continue     fine=rules[offence]+(500 if
repeat=="yes" else 0)     total+=fine     if
offence=="signal" or total>=3000:
        disqualified=True
print("Vehicle:",vehicle)
print("Total penalty: Rs.",total)
print("Disqualification:","YES" if disqualified else "NO")
```

8. TEST CASE

Sample: car; speeding once and repeat signal offence.

9. CODE AND OUTPUT SCREENSHOT

Code

code1.py

```
1  rules={"speeding":1000,"signal":1500,"no_helmet":500}
2  total=0
3  disqualified=False
4  vehicle=input("Vehicle (car/bike): ").lower()
5  n=int(input("Number of offences: "))
6  for i in range(n):
7      offence=input("Offence: ").lower()
8      repeat=input("Repeat? (yes/no): ").lower()
9      if offence not in rules:
10         print("Invalid offence; skipped")
11         continue
12     fine=rules[offence]+(500 if repeat=="yes" else 0)
13     total+=fine
14     if offence=="signal" or total>=3000:
15         disqualified=True
16 print("Vehicle:",vehicle)
17 print("Total penalty: Rs.",total)
18 print("Disqualification:","YES" if disqualified else "NO")
```

Output

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python ABL
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/code1.py"
Vehicle (car/bike): car
Number of offences: 2
Offence: speeding
Repeat? (yes/no): no
Invalid offence; skipped
Offence: signal
Repeat? (yes/no): yes
Vehicle: car
Total penalty: Rs. 2000
Disqualification: YES
```

10. CONCLUSION

Program calculates total fines and checks the sample disqualification rule

QUESTION 22: Laboratory Chemical Inventory Validation

1. PROBLEM STATEMENT

Validate chemical category and quantity records and flag a sample incompatible pair.

2. OBJECTIVES

Read chemical name, category and quantity • Check category and maximum quantity • Flag unknown or invalid records • Check incompatible pair • Display report

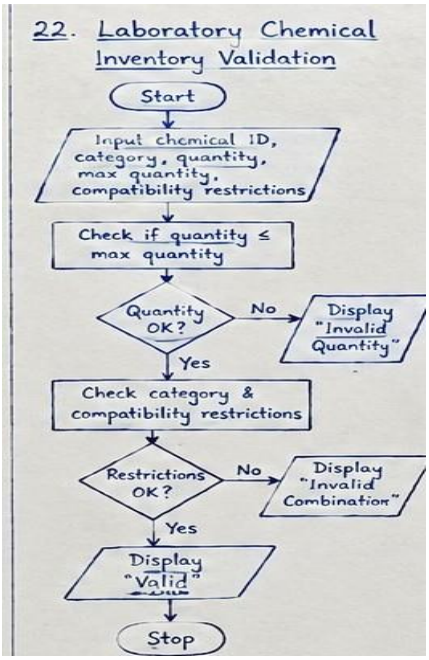
3. ASSUMPTIONS

acid: corrosive, max 10 L; bleach: oxidizer, max 5 L; ethanol: flammable, max 8 L. Acid + bleach triggers sample warning.

4. ALGORITHM

1. Start → 2. Input chemical records → 3. Check known chemical/category → 4. Quantity within maximum? → 5. Mark valid or invalid → 6. Acid and bleach both present? → 7. Display report/warning → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input chemical records
```

```
3. Check known chemical/category
4. Quantity within maximum?
5. Mark valid or invalid
6. Acid and bleach both present?
7. Display report/warning
8. Stop
END
```

7. PYTHON SOURCE CODE

```
rules={"acid":("corrosive",10),"bleach":("oxidizer",5),"ethanol":("flammable",8)}
records=[] n=int(input("Number of records: ")) for i in range(n):
    name=input("Chemical: ").lower()
cat=input("Category: ").lower()
qty=float(input("Quantity (L): "))
records.append((name,cat,qty)) for
name,cat,qty in records:
    if name not in rules: print(name,"UNKNOWN")     elif
cat!=rules[name][0] or qty>rules[name][1]: print(name,"INVALID")     else:
print(name,"VALID") names=[r[0] for r in records] if "acid" in names and
"bleach" in names:     print("WARNING: acid and bleach are incompatible")
```

8. TEST CASE

Sample: acid 4 L and bleach 2 L, with correct categories.

9. CODE AND OUTPUT SCREENSHOT

Code

code2.py > ...

```
1  rules={"acid":("corrosive",10),"bleach":("oxidizer",5),"ethanol":("flammable",8)}
2  records=[]
3  n=int(input("Number of records: "))
4  for i in range(n):
5      name=input("Chemical: ").lower()
6      cat=input("Category: ").lower()
7      qty=float(input("Quantity (L): "))
8      records.append((name,cat,qty))
9  for name,cat,qty in records:
10     if name not in rules: print(name,"UNKNOWN")
11     elif cat!=rules[name][0] or qty>rules[name][1]: print(name,"INVALID")
12     else: print(name,"VALID")
13 names=[r[0] for r in records]
14 if "acid" in names and "bleach" in names:
15     print("WARNING: acid and bleach are incompatible")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python A
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/code2.py"
Number of records: 3
Chemical: acid
Category: corrosive
Quantity (L): 5
Chemical: bleach
Category: oxidizer
Quantity (L): 2
Chemical: ethanol
Category: flammable
Quantity (L): 10
acid VALID
bleach VALID
ethanol INVALID
WARNING: acid and bleach are incompatible
```

10. CONCLUSION

Program validates sample records and flags the specified incompatible pair.

QUESTION 23: Student Timetable Conflict Detection

1. PROBLEM STATEMENT

Detect overlapping course periods and courses assigned to unavailable rooms.

2. OBJECTIVES

Load course schedule and unavailable rooms • Compare every pair of classes • Detect overlap on the same day • Check unavailable rooms • Display timetable status

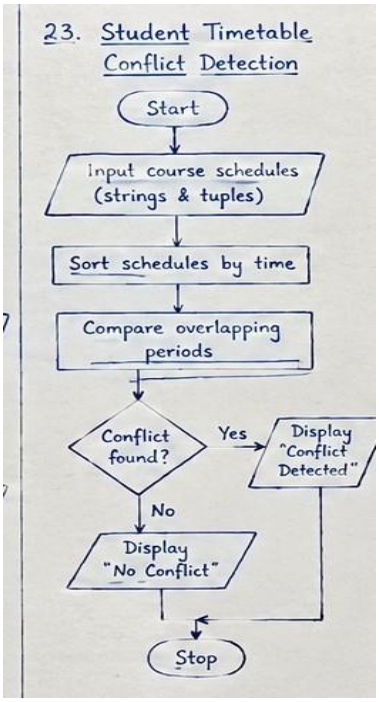
3. ASSUMPTIONS

Times are minutes after midnight. Two periods overlap if each starts before the other ends. R102 is unavailable in the sample.

4. ALGORITHM

1. Start → 2. Load schedules and unavailable rooms → 3. Compare same-day periods → 4. Periods overlap? → 5. Report time conflict → 6. Room unavailable? → 7. Display timetable status → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Load schedules and unavailable rooms
3. Compare same-day periods
```

```
4. Periods overlap?
5. Report time conflict
6. Room unavailable?
7. Display timetable status
8. Stop
END
```

7. PYTHON SOURCE CODE

```
classes=[("Python","Mon",540,600,"R101"),("Maths","Mon",590,650,"R102"),("English","Tue",600,660,"R103")]
unavailable=["R102"] bad=False for i in range(len(classes)):     for j in range(i+1,len(classes)):
a,b=classes[i],classes[j]          if a[1]==b[1] and a[2]<b[3] and b[2]<a[3]:
            print("Time conflict:",a[0],"and",b[0]); bad=True
for course,day,start,end,room in classes:     if room in
unavailable:
        print("Unavailable room:",room,"for",course); bad=True
print("Timetable is","NOT valid" if bad else "valid")
```

8. TEST CASE

Sample: Python 09:00–10:00 and Maths 09:50–10:50 Monday; R102 unavailable.

9. CODE AND OUTPUT SCREENSHOT

Code

c3.py > ... Extension: Pylance

```
1  classes=[("Python","Mon",540,600,"R101"),("Maths","Mon",590,650,"R102")
2           ,("English","Tue",600,660,"R103")]
3  unavailable=["R102"]
4  bad=False
5  for i in range(len(classes)):
6      for j in range(i+1,len(classes)):
7          a,b=classes[i],classes[j]
8          if a[1]==b[1] and a[2]<b[3] and b[2]<a[3]:
9              print("Time conflict:",a[0],"and",b[0]); bad=True
10 for course,day,start,end,room in classes:
11     if room in unavailable:
12         print("Unavailable room:",room,"for",course); bad=True
13 print("Timetable is","NOT valid" if bad else "valid")
```

Output

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c3.py"
Time conflict: Python and Maths
Unavailable room: R102 for Maths
Timetable is NOT valid
```

10. CONCLUSION

Program reports overlapping classes and unavailable rooms.

QUESTION 24: Courier Charge Optimization

1. PROBLEM STATEMENT

Calculate sample courier prices and select the cheapest eligible service.

2. OBJECTIVES

Read parcel details • Calculate volume • Check each service's limits • Calculate eligible charges • Display cheapest service

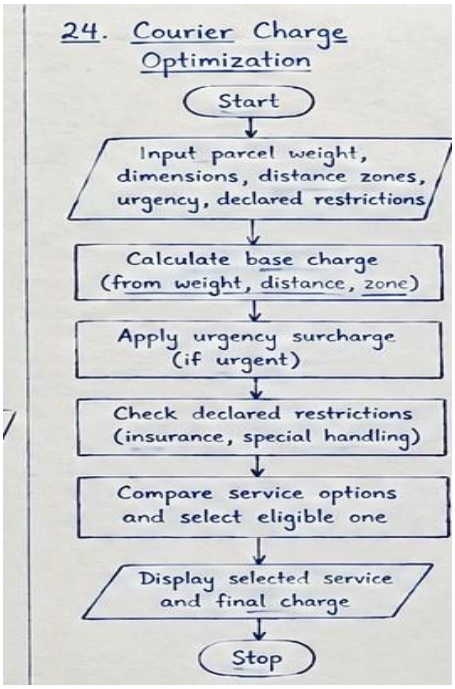
3. ASSUMPTIONS

Sample rates: zone fees Rs.50/Rs.100/Rs.150; Rs.20 per kg; urgent surcharge Rs.100. Standard base Rs.40 (10 kg/50000 cm^3); Express base Rs.80 (5 kg/30000 cm^3). Restricted parcels rejected.

4. ALGORITHM

1. Start → 2. Input parcel details → 3. Calculate volume → 4. Check limits and restrictions → 5. Service eligible? → 6. Calculate charges and compare → 7. Display cheapest service → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input parcel details
3. Calculate volume
4. Check limits and restrictions
```

```
5. Service eligible?
6. Calculate charges and compare
7. Display cheapest service
8. Stop
END
```

7. PYTHON SOURCE CODE

```
weight=float(input("Weight kg: ")) l=float(input("Length cm: ")); w=float(input("Width cm:
")); h=float(input("Height cm: ")) zone=int(input("Zone (1-3): ")) urgent=input("Urgent?
(yes/no): ").lower() restricted=input("Restricted? (yes/no): ").lower()
fees={1:50,2:100,3:150}; services={"Standard":(10,50000,40),"Express":(5,30000,80)}
options=[]; volume=l*w*h if zone in fees and restricted=="no":
    for name,(mw,mv,base) in services.items():         if weight<=mw and
volume<=mv:             cost=base+weight*20+fees[zone]+(100 if
urgent=="yes" else 0)             options.append((cost,name)) if options:
    options.sort()     for cost,name in options:
print(name,"charge: Rs.",cost)     print("Cheapest
service:",options[0][1]) else: print("No eligible service")
```

8. TEST CASE

Sample: 2 kg, 20×10×10 cm, zone 1, not urgent/restricted.

9. CODE AND OUTPUT SCREENSHOT

Code

c4.py > ...

```python
weight=float(input("Weight kg: "))
l=float(input("Length cm: ")); w=float(input("Width cm: ")); h=float(input("Height cm: "))
zone=int(input("Zone (1-3): "))
urgent=input("Urgent? (yes/no): ").lower()
restricted=input("Restricted? (yes/no): ").lower()
fees={1:50,2:100,3:150}; services={"Standard":(10,50000,40),"Express":(5,30000,80)}
options=[]; volume=l*w*h
if zone in fees and restricted=="no":
    for name,(mw,mv,base) in services.items():
        if weight<=mw and volume<=mv:
            cost=base+weight*20+fees[zone]+(100 if urgent=="yes" else 0)
            options.append((cost,name))
if options:
    options.sort()
    for cost,name in options: print(name,"charge: Rs.",cost)
    print("Cheapest service:",options[0][1])
else: print("No eligible service")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c4.py"
Weight kg: 2
Length cm: 20
Width cm: 10
Height cm: 10
Zone (1-3): 1
Urgent? (yes/no): no
Restricted? (yes/no): no
Standard charge: Rs. 130.0
Express charge: Rs. 170.0
Cheapest service: Standard
```

10. CONCLUSION

Program compares eligible sample services and selects the lowest charge.

QUESTION 25: Progressive Water Tariff Analysis

1. PROBLEM STATEMENT

Calculate household water bills using progressive slabs, fixed fees and a sample penalty.

2. OBJECTIVES

Read each household's consumption • Apply each tariff slab progressively • Add fixed fee and penalty if needed • Display each bill • Find highest consumption

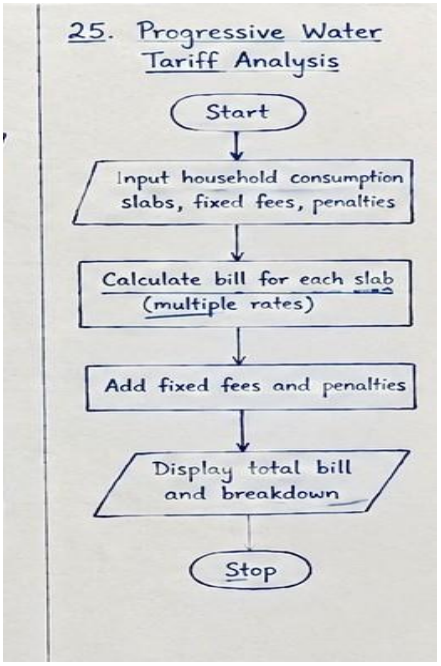
3. ASSUMPTIONS

Fixed fee Rs.50; first 10 units Rs.2/unit; next 10 units Rs.3/unit; above 20 units Rs.5/unit; usage above 30 units adds Rs.100. Sample rates only.

4. ALGORITHM

1. Start → 2. Input household and units → 3. Calculate slab-wise charge → 4. Add fixed fee/penalty → 5. Display bills → 6. Find highest usage → 7. Display highest consumption → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input household and units
3. Calculate slab-wise charge
4. Add fixed fee/penalty
```

```
5. Display bills
6. Find highest usage
7. Display highest consumption
8. Stop
END
```

7. PYTHON SOURCE CODE

```
def bill(u):
    total=50+min(u,10)*2     if
u>10: total+=min(u-10,10)*3     if
u>20: total+=(u-20)*5     if u>30:
total+=100     return total
homes=[] n=int(input("Households:
")) for i in range(n):
    name=input("Name: "); units=float(input("Units: "))
homes.append((name,units,bill(units))) for
name,units,amount in homes:
    print(name,"Units:",units,"Bill: Rs.",amount)
if units>30: print("Penalty applied") if homes:
    top=max(homes,key=lambda x:x[1])
    print("Highest consumption:",top[0],top[1],"units")
```

8. TEST CASE

Sample: Asha uses 8 units; Ravi uses 35 units.

9. CODE AND OUTPUT SCREENSHOT

Code

c5.py > ...

```
1   def bill(u):
2       total=50+min(u,10)*2
3       if u>10: total+=min(u-10,10)*3
4       if u>20: total+=(u-20)*5
5       if u>30: total+=100
6       return total
7   homes=[]
8   n=int(input("Households: "))
9   for i in range(n):
10      name=input("Name: "); units=float(input("Units: "))
11      homes.append((name,units,bill(units)))
12  for name,units,amount in homes:
13      print(name,"Units:",units,"Bill: Rs.",amount)
14      if units>30: print("Penalty applied")
15  if homes:
16      top=max(homes,key=lambda x:x[1])
17      print("Highest consumption:",top[0],top[1],"units")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\P
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c5
Households: 2
Name: geethu
Units: 8
Name: arjun
Units: 35
geethu Units: 8.0 Bill: Rs. 66.0
arjun Units: 35.0 Bill: Rs. 275.0
Penalty applied
Highest consumption: arjun 35.0 units
```

10. CONCLUSION

Program calculates progressive bills, applies the sample penalty and identifies highest use.

QUESTION 26: Nested Loop Examination Seating Generator

1. PROBLEM STATEMENT

Generate a seating chart with row/column labels, reserved seats and alternate empty seats.

2. OBJECTIVES

Read rows, columns and alternate rule • Use nested loops to visit seats • Mark reserved seats • Mark alternate seats empty if required • Display seating chart

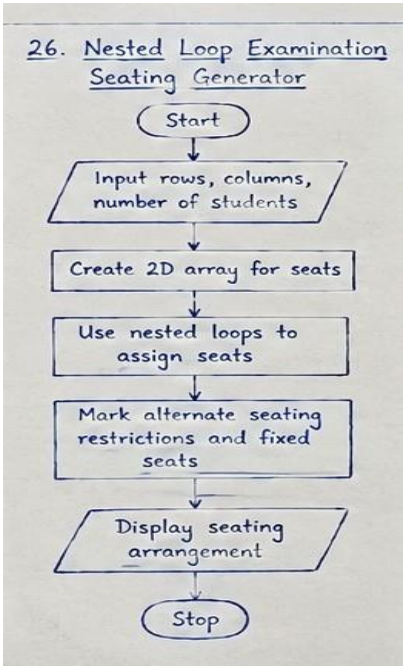
3. ASSUMPTIONS

Row and column counts must be positive. Seat (1,2) is reserved in the sample. Alternate rule leaves even columns empty except reserved seats.

4. ALGORITHM

1. Start → 2. Input rows, columns and rule → 3. Visit seats using nested loops → 4. Seat reserved? → 5. Mark reserved seat → 6. Alternate seat empty? → 7. Mark empty/normal seat → 8. Display seating chart → 9. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input rows, columns and rule
3. Visit seats using nested loops
4. Seat reserved?
```

```
5. Mark reserved seat
6. Alternate seat empty?
7. Mark empty/normal seat
8. Display seating chart
9. Stop
END
```

7. PYTHON SOURCE CODE

```
rows=int(input("Rows: ")); cols=int(input("Seats per row: "))
alternate=input("Alternate seats empty? (yes/no): ").lower()
reserved={(1,2)} if rows<=0 or cols<=0:
    print("Rows and columns must be positive")
else:     for r in range(1,rows+1):
for c in range(1,cols+1):
            if (r,c) in reserved: seat="RES"
elif alternate=="yes" and c%2==0: seat="EMPTY"
else: seat=f"R{r}C{c}"
print(f"{seat:8}",end="")         print()
```

8. TEST CASE

Sample: 2 rows, 3 seats each, alternate seating enabled.

9. CODE AND OUTPUT SCREENSHOT

Code

c6.py > ... C:\Users\HP\OneDrive\Desktop\CSE-1\pytho

```
1  rows=int(input("Rows: ")); cols=int(input("Seats per row: "))
2  alternate=input("Alternate seats empty? (yes/no): ").lower()
3  reserved={(1,2)}
4  if rows<=0 or cols<=0:
5      print("Rows and columns must be positive")
6  else:
7      for r in range(1,rows+1):
8          for c in range(1,cols+1):
9              if (r,c) in reserved: seat="RES"
10             elif alternate=="yes" and c%2==0: seat="EMPTY"
11             else: seat=f"R{r}C{c}"
12             print(f"{seat:8}",end="")
13         print()
```

Output

Problems Output Debug Console Terminal Po

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python
eDrive/Desktop/CSE-1/python lab/Python ABL S
Rows: 2
Seats per row: 3
Alternate seats empty? (yes/no): yes
R1C1    RES     R1C3
R2C1    EMPTY   R2C3
```

10. CONCLUSION

Program displays the seating plan while applying reserved and alternate seating rules.

QUESTION 27: Advanced Numeric Pattern Generator

1. PROBLEM STATEMENT

Validate a size and generate a symmetric hollow diamond pattern.

2. OBJECTIVES

Read the diamond size • Reject non-positive size • Print upper hollow-diamond rows • Print lower rows in reverse • Display pattern

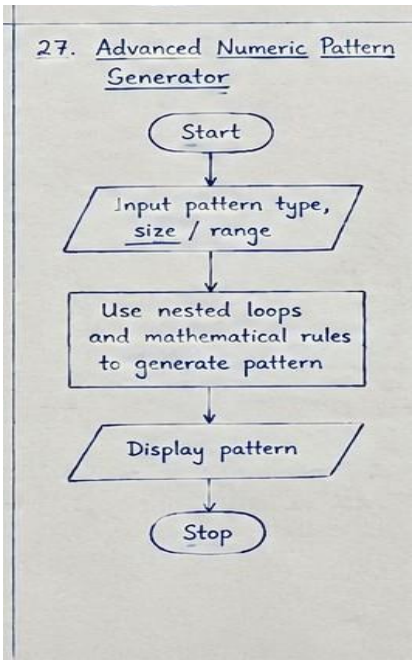
3. ASSUMPTIONS

This solution prints a hollow star diamond as one valid interpretation of the pattern task. Size must be a positive integer.

4. ALGORITHM

1. Start → 2. Input diamond size → 3. Is size positive? → 4. If no, display error → 5. Print upper half → 6. Print lower half → 7. Display diamond → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input diamond size
3. Is size positive?
4. If no, display error
5. Print upper half
```

```
6. Print lower half
7. Display diamond
8. Stop
END
```

7. PYTHON SOURCE CODE

```
n=int(input("Diamond size: "))
if n<=0:
    print("Enter a positive size")
else:     for i in range(1,n+1):
        s=" "*(n-i)          print(s+"*" if i==1 else
s+"*"+" "*(2*i-3)+"*")      for i in range(n-1,0,-1):
        s=" "*(n-i)
        print(s+"*" if i==1 else s+"*"+" "*(2*i-3)+"*")
```

8. TEST CASE

Sample: diamond size 3.

9. CODE AND OUTPUT SCREENSHOT

Code

c7.py > ...

```
1  n=int(input("Diamond size: "))
2  if n<=0:
3      print("Enter a positive size")
4  else:
5      for i in range(1,n+1):
6          s=" "*(n-i)
7          print(s+"*" if i==1 else s+"*"+" "*(2*i-3)+"*")
8      for i in range(n-1,0,-1):
9          s=" "*(n-i)
10         print(s+"*" if i==1 else s+"*"+" "*(2*i-3)+"*")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c7.py"
Diamond size: 3
  *
 * *
*   *
 * *
  *
```

10. CONCLUSION

Program validates the size and prints a symmetric hollow diamond.

QUESTION 28: Repeated Transaction Detection

1. PROBLEM STATEMENT

Detect duplicate transaction IDs and amounts above a sample threshold.

2. OBJECTIVES

Load transaction IDs and amounts • Track IDs already seen • Flag repeated IDs • Flag amounts above threshold • Display suspicious transactions

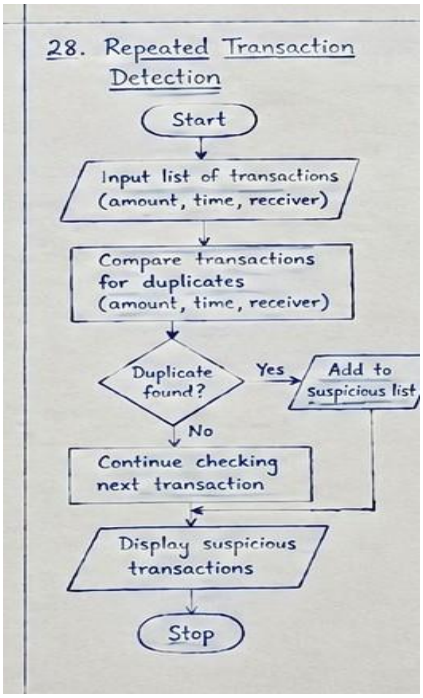
3. ASSUMPTIONS

Sample transactions are included in the code. Threshold is Rs.2000. An item is suspicious if its ID repeats or its amount exceeds the threshold.

4. ALGORITHM

1. Start → 2. Load transaction list → 3. Check ID in seen set → 4. Duplicate ID? → 5. Flag duplicate → 6. Amount above threshold? → 7. Display suspicious transactions → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Load transaction list
```

```
3. Check ID in seen set
4. Duplicate ID?
5. Flag duplicate
6. Amount above threshold?
7. Display suspicious transactions
8. Stop
END
```

7. PYTHON SOURCE CODE

```
transactions=[("T101",500),("T102",2500),("T101",500),("T103",1200)]
threshold=2000 seen=set() for tid,amount in transactions:
    reasons=[]     if tid in seen: reasons.append("duplicate ID")
else: seen.add(tid)     if amount>threshold:
reasons.append("amount above threshold")     if reasons:
print(tid,"Rs.",amount,"-",", ".join(reasons))
```

8. TEST CASE

Sample transactions include repeated T101 and T102 with amount Rs.2500.

9. CODE AND OUTPUT SCREENSHOT

Code

c8.py > ...

```
1  transactions=[("T101",500),("T102",2500),("T101",500),("T103",1200)]
2  threshold=2000
3  seen=set()
4  for tid,amount in transactions:
5      reasons=[]
6      if tid in seen: reasons.append("duplicate ID")
7      else: seen.add(tid)
8      if amount>threshold: reasons.append("amount above threshold")
9      if reasons: print(tid,"Rs.",amount,"-",", ".join(reasons))
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python ABL SEM1>
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c8.py"
T102 Rs. 2500 - amount above threshold
T101 Rs. 500 - duplicate ID
```

10. CONCLUSION

Program flags duplicate transaction IDs and amounts above the sample threshold.

QUESTION 29: Password Policy Validation Engine

1. PROBLEM STATEMENT

Check password length, character categories, repetition and prohibited text.

2. OBJECTIVES

Read sample password • Check minimum length and character groups • Check three repeated characters • Check prohibited substring • Display failures or valid status

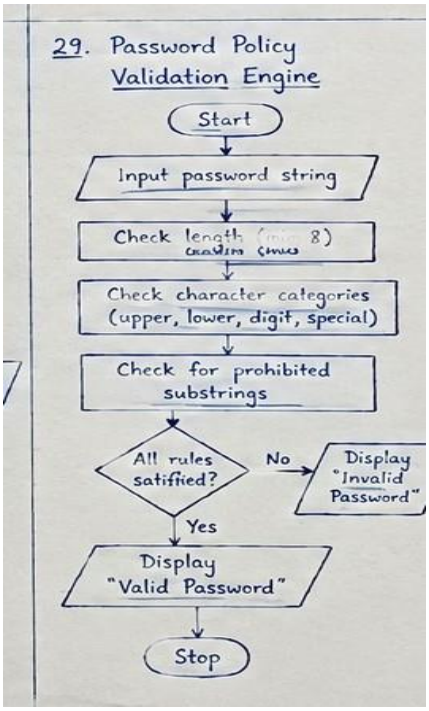
3. ASSUMPTIONS

Sample rules: minimum 8 characters, uppercase, lowercase, digit and special character required; three identical consecutive characters and the word “password” are prohibited.

4. ALGORITHM

1. Start → 2. Input password → 3. Check length and character groups → 4. Check repeated characters → 5. Check prohibited substring → 6. All rules satisfied? → 7. Display status → 8. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input password
```

```
3. Check length and character groups
4. Check repeated characters
5. Check prohibited substring
6. All rules satisfied?
7. Display status
8. Stop
END
```

7. PYTHON SOURCE CODE

```
p=input("Sample password: ") failed=[] if len(p)<8: failed.append("Minimum
length is 8") if not any(c.isupper() for c in p): failed.append("Add
uppercase") if not any(c.islower() for c in p): failed.append("Add lowercase")
if not any(c.isdigit() for c in p): failed.append("Add digit") if not any(not
c.isalnum() for c in p): failed.append("Add special character") if
any(p[i]==p[i+1]==p[i+2] for i in range(len(p)-2)):
    failed.append("Avoid 3 repeated characters") if "password" in
p.lower(): failed.append("Avoid word password") if failed:
    print("Password does not meet policy:")
for item in failed: print("-",item) else:
print("Password meets sample policy")
```

8. TEST CASE

Sample password: abc.

9. CODE AND OUTPUT SCREENSHOT

Code

c9.py > ...

```
1  p=input("Sample password: ")
2  failed=[]
3  if len(p)<8: failed.append("Minimum length is 8")
4  if not any(c.isupper() for c in p): failed.append("Add uppercase")
5  if not any(c.islower() for c in p): failed.append("Add lowercase")
6  if not any(c.isdigit() for c in p): failed.append("Add digit")
7  if not any(not c.isalnum() for c in p): failed.append("Add special character")
8  if any(p[i]==p[i+1]==p[i+2] for i in range(len(p)-2)):
9      failed.append("Avoid 3 repeated characters")
10 if "password" in p.lower(): failed.append("Avoid word password")
11 if failed:
12     print("Password does not meet policy:")
13     for item in failed: print("-",item)
14 else: print("Password meets sample policy")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python ABL SEM1> &
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c9.py"
Sample password: geethu
Password does not meet policy:
- Minimum length is 8
- Add uppercase
- Add digit
- Add special character
```

10. CONCLUSION

Program lists all sample password rules that are not satisfied.

QUESTION 30: Student Email and Identifier Validation

1. PROBLEM STATEMENT

Validate an email-like address and student registration ID using basic formatting rules.

2. OBJECTIVES

Read email and student ID • Check exactly one @ symbol • Check username and domain • Check student ID characters • Display errors or valid status

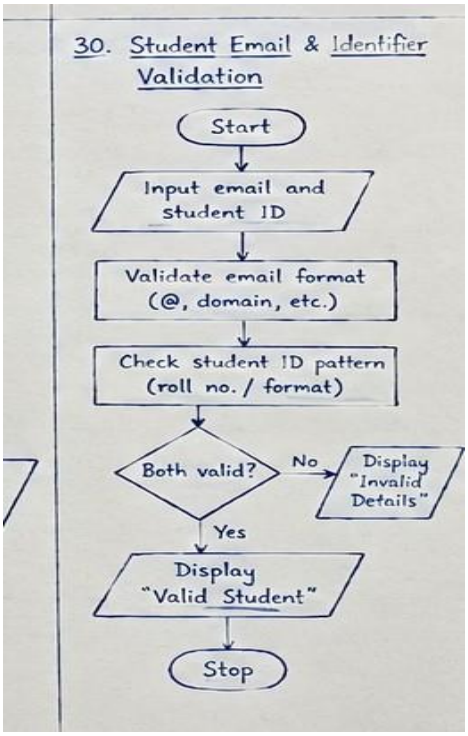
3. ASSUMPTIONS

Email requires exactly one @, a username, and a domain containing a dot; spaces are rejected. Student ID allows letters, digits and hyphens. Format checking does not prove existence.

4. ALGORITHM

1. Start → 2. Input email and student ID → 3. Validate email format → 4. Validate ID pattern → 5. Both valid? → 6. Display result → 7. Stop

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
1. Start
2. Input email and student ID
```

```
3. Validate email format
4. Validate ID pattern
5. Both valid?
6. Display result
7. Stop
END
```

7. PYTHON SOURCE CODE

```
email=input("Student email: ").strip() sid=input("Student ID: ").strip()
errors=[] if email.count("@")!=1: errors.append("Email must contain
exactly one @") else:
    user,domain=email.split("@")     if not user:
errors.append("Username missing")     if not domain or "." not in
domain: errors.append("Invalid domain")     if " " in email:
errors.append("Email must not contain spaces") if not sid:
errors.append("Student ID missing") elif not all(c.isalnum() or c=="-"
for c in sid):     errors.append("Invalid student ID characters") if
errors:
    print("Invalid details:")     for e in
errors: print("-",e) else: print("Email and
student ID are valid")
```

8. TEST CASE

Sample: geetha@example.com and CSE-102.

9. CODE AND OUTPUT SCREENSHOT

Code

c10.py > ...

```
1  email=input("Student email: ").strip()
2  sid=input("Student ID: ").strip()
3  errors=[]
4  if email.count("@")!=1: errors.append("Email must contain exactly one @")
5  else:
6      user,domain=email.split("@")
7      if not user: errors.append("Username missing")
8      if not domain or "." not in domain: errors.append("Invalid domain")
9      if " " in email: errors.append("Email must not contain spaces")
10 if not sid: errors.append("Student ID missing")
11 elif not all(c.isalnum() or c=="-" for c in sid):
12     errors.append("Invalid student ID characters")
13 if errors:
14     print("Invalid details:")
15     for e in errors: print("-",e)
16 else: print("Email and student ID are valid")
```

Output

Problems Output Debug Console Terminal Ports

```
PS C:\Users\HP\OneDrive\Desktop\CSE-1\python lab\Python ABL SEM1>
eDrive/Desktop/CSE-1/python lab/Python ABL SEM1/c10.py"
Student email: geethu132009@gmail.com
Student ID: CSE-125
Email and student ID are valid
```

10. CONCLUSION

Program validates basic email and student-ID formatting.